CS2002: Mid-Sem Solution

Question 1

Let the original problem be:

$A = \begin{bmatrix} -25 & -25 \end{bmatrix}$
With target-sum $W = -50$

The negative number with highest magnitude is -25 so we will add $+25$ to all the elements of array A. So the new array becomes:

$A = \begin{bmatrix} 0 & 0 \end{bmatrix}$

And to compensate for the $25's$ we add to the elements, we must increase the target sum as we increase the size of the sliding window. That is, the targets for window of size {1, 2} will be {-25, 0} respectively.

$$A = \begin{bmatrix} \boxed{0} & 0 \end{bmatrix} \implies \boxed{0 > -25}$$

$$A = \begin{bmatrix} 0 & \boxed{0} \end{bmatrix} \implies \boxed{0 > -25}$$

Here the sliding window algorithm terminates and fails to find any subarray.
But we know that:

$$A = \begin{bmatrix} -25 & -25 \end{bmatrix} \implies \boxed{-25 + (-25) = -50}$$

Therefore, we can conclude that this approach does not work as intended.

Evaluation Scheme

- Correct verdict (stating "No" / algorithm fails) 1 Mark
- Valid justification / counterexample 1 Mark

Question 2

Algorithms

Algorithm 1 Find: p-parent

```
1: procedure FIND(i)
2:     if p[i] == i then
3:         return i
4:     end if
5:     p[i] ← FIND(p[i])
6:     return p[i]
7: end procedure
```

Algorithm 2 Build: p-parent, r-rank

```
1: procedure BUILD(u, v)
2:     rep_u ← FIND(u)
3:     rep_v ← FIND(v)
4:     if rep_u == rep_v then
5:         return
6:     end if
7:     if r[rep_u] > r[rep_v] then
8:         p[rep_v] ← p[rep_u]
9:         return
10:    end if
11:    if r[rep_u] < r[rep_v] then
12:        p[rep_u] ← p[rep_v]
13:        return
14:    end if
15:    p[rep_v] ← rep_u
16:    r[rep_u] ← r[rep_u] + 1
17: end procedure
```

Algorithm 3 Connected

```
1: procedure CONNECTED(u, v)
2:     if FIND(u) == FIND(v) then
3:         return 1
4:     else
5:         return 0
6:     end if
7: end procedure
```

Time Complexity Analysis

- **Initialisation:** Initialising n cities will take $O(n)$ time.
- **Union by Rank alone (using the concept of rank:**
  Find: worst-case running time of - $O(logn)$.
  Connected: uses Find and constant time comparision - $O(logn)$.
  Build: uses Find and constant time comparision and assignment operation - $O(logn)$.

- **Using rank and path-compression: (over m operations):**
  Find: worst-case running time of - $O(m\alpha(n))$.
  Connected: uses Find and constant time comparision - $O(m\alpha(n))$.
  Build: uses Find and constant time comparision and assignment operation - $O(m\alpha(n))$.

Evaluation Scheme

- **Efficient Approach (Disjoint Set Union / DSU):** **[Max 3 Marks]**
  - Mentioning the data structure — 1 Mark
  - Correct implementation/pseudocode for $\textsc{Build}(u, v)$ — 1 Mark
  - Correct implementation/pseudocode for $\textsc{Connected}(u, v)$ — 1 Mark
- **Inefficient Approach (Other representations, e.g., Graph/BFS/DFS): [Max 1.5 Marks]**
  - Mentioning the data structure — 0.5 Mark
  - Implementation/pseudocode for $\textsc{Build}(u, v)$ — 0.5 Mark
  - Implementation/pseudocode for $\textsc{Connected}(u, v)$ — 0.5 Mark
- **Inefficient approach for one of Build(u, v), Connected(u, v), will still be eligible for full marks a long as efficient data structure is mentioned.**

Question 3

1 mark for state representation if the student clearly maintains the current integer to be assigned together with the current size and sum of each of the m groups (or an equivalent sufficient representation);
1 mark for the branching strategy if the student tries assigning the next integer to each feasible group and recursively explores the resulting states with backtracking;
1 mark for pruning if the student discards a branch when a group contains more than three integers, its sum exceeds the target sum $B = \frac{\sum_i a_i}{m}$, or an equivalent infeasibility condition is detected; and
1 mark for the worst-case running-time analysis, with $O(m^{3m})$ being an acceptable bound since there are $3m$ stages and at most m choices at each stage.

Question 4

Model Solution: Sum of the Five Highest Marks

[4 Marks]

Throughout, assume $n \geq 5$. The marks $A[1], \ldots, A[n]$ may be any real numbers.

1. Definition of the DP table [1 mark]

For each i from 0 to n, let $dp[i]$ be an array of five cells that stores the five largest marks among $A[1], \ldots, A[i]$. When i is less than 5 there are not yet five marks to store, so the unused cells hold $-\infty$. The order of the five cells does not matter. The table has $(n+1) \times 5$ cells.

2. Initialization [0.5 marks]

$$dp[0][t] = -\infty \qquad \text{for } t = 1, 2, 3, 4, 5.$$

3. Recursion [1.5 marks]

For $i = 1$ to n, let m be the smallest value in $dp[i-1]$, held in cell p. Then

$$dp[i] = \begin{cases} dp[i-1], & \text{if } A[i] \leq m, \\ dp[i-1] \text{ with cell } p \text{ changed to } A[i], & \text{if } A[i] > m. \end{cases}$$

In words: if the new mark is not bigger than the smallest of the five stored so far, keep the row unchanged. Otherwise the new mark belongs in the top five, so it replaces the smallest one. As pseudocode (optional, not required for full marks):

```
for i = 1 to n:
    dp[i] ← dp[i − 1]
    p ← index of the smallest value in dp[i]
    if A[i] > dp[i][p]:
        dp[i][p] ← A[i]
```

4. Output of the top five marks [1 mark]

The five highest marks are the five values in the last row,

$$dp[n][1], \quad dp[n][2], \quad dp[n][3], \quad dp[n][4], \quad dp[n][5],$$

and the required sum is

$$dp[n][1] + dp[n][2] + dp[n][3] + dp[n][4] + dp[n][5].$$

No backtracking through the table is needed, because the marks themselves are stored in the row. If they are wanted in decreasing order, sort these five values at the end.

Running time and space [not required for marks]

The four parts above carry the full 4 marks. The following is stated only to confirm that the algorithm meets the bounds asked for in the question, and no marks depend on it. Every row of the table has exactly five cells, whatever the value of n. So one step of the recursion copies five values, makes four comparisons to find the smallest one, makes one more comparison against $A[i]$, and writes at most one cell. That is a fixed amount of work that does not grow with n, so each step takes constant time. There are n such steps, one for each student, and reading off the answer at the end costs another constant amount of work. The total running time is therefore $O(n)$. The table stores $5(n+1)$ numbers, so the space is $O(n)$. Only the previous row is ever used, so a single array of five cells would also do, which uses $O(1)$ extra space and is well within the required bound.

Question 5

Dynamic Programming for the k Knapsack Problem

Marks splitup

- 1 mark: DP state
- 1 mark: Recurrence + base case
- 1 mark: Recovering the solution
- 1 mark: Runtime + space

Problem

We are given n objects with weights $w_1, w_2, \ldots, w_n$ and a knapsack of capacity W. The objective is to select exactly k objects such that their total weight does not exceed W, while maximizing the total weight of the selected objects.

1. DP State

Let $DP[i][j][c]$ denote the maximum total weight obtainable by selecting exactly j objects from the first i objects, with total weight at most c, where, $0 \leq i \leq n, \qquad 0 \leq j \leq k, \qquad 0 \leq c \leq W.$ If a state is impossible, we set its value to $-\infty$.

2. Initialization

With zero objects, we can select exactly zero objects with total weight 0:

$$DP[0][0][c] = 0 \qquad \text{for all } 0 \leq c \leq W.$$

Selecting a positive number of objects from zero objects is impossible:

$$DP[0][j][c] = -\infty \qquad \text{for } j > 0.$$

3. Recurrence

For object i, there are two choices.

Case 1: Do not select object i

$$DP[i][j][c] = DP[i-1][j][c].$$

Case 2: Select object i

If $w_i \leq c$, then we select object i and need to select $j-1$ objects from the first $i-1$ objects:

$$DP[i][j][c] = DP[i-1][j-1][c-w_i] + w_i.$$

Therefore, when $w_i \leq c$,

$$\boxed{DP[i][j][c] = \max\left\{DP[i-1][j][c], DP[i-1][j-1][c-w_i] + w_i\right\}.}$$

If $w_i > c$, object i cannot be selected, and hence

$$DP[i][j][c] = DP[i-1][j][c].$$

The required optimal value is

$$\boxed{DP[n][k][W]}.$$

4. Recovering the Selected Objects

The selected objects can be recovered directly from the same DP table. No additional parent or decision table is required.

Start from the final state $(i, j, c) = (n, k, W)$. At each step, compare $DP[i][j][c]$ with $DP[i-1][j][c]$.

- If $DP[i][j][c] = DP[i-1][j][c]$, then object i was not selected. Move to
$$(i-1, j, c).$$
- Otherwise, object i was selected. Add object i to the solution and move to
$$(i-1, j-1, c-w_i).$$

Continue until $j = 0$. The collected objects form an optimal set of exactly k objects.

5. Complexity Analysis

There are
$$(n+1)(k+1)(W+1) = O(nkW)$$
DP states, and each state can be computed in $O(1)$ time. Since the complete DP table is stored in order to recover the selected objects, the space complexity is

$$\boxed{O(nkW)}.$$

Question 6

Greedy Algorithm [1 Mark]

Input: Starting position ($x_0 = 0$), destination D, a list of intermediate stations X, and the maximum distance L the vehicle can travel on a full charge.

Output: The minimum number of charging stops required to reach the destination, or a report that the journey is impossible.

Algorithm:

1. Append the starting position and the destination to the beginning and end of the list of stations, respectively.
2. Let $X = [x_0, x_1, x_2, \ldots, x_n, x_{n+1}]$ where $x_0 = 0$ and $x_{n+1} = D$.
3. Check for impossibility: Iterate through X. If the distance between any two consecutive points $x_i - x_{i-1} > L$ for $1 \leq i \leq n+1$, return **"Impossible"**.
4. Initialize $\mathit{current_position} = 0$, $\mathit{stops} = 0$, and an index $i = 0$.
5. While $D - \mathit{current_position} > L$:
   - Increment index i until the distance to the next station exceeds L: while $x_{i+1} - \mathit{current_position} \leq L$, do $i = i + 1$.
   - Set $\mathit{current_position} = x_i$.
   - Increment $\mathit{stops} = \mathit{stops} + 1$.
6. Return *stops*.

Time Complexity: $\mathcal{O}(n)$ because the algorithm iterates through the stations at most once.

Greedy Safeness Property & Proof [2 Marks]

Greedy Choice:
Pick the furthest possible station the vehicle can reach on its current charge.

Greedy-Safeness Property: [1 Mark]
Let x_k be the furthest charging station reachable from the current position. There exists an optimal solution that includes a stop at x_k.

Proof (By Exchange Argument): [1 Mark]

- Let S be an optimal sequence of charging stops.
- Suppose the first stop in S is at a station s_j.
- If $s_j = x_k$, our greedy choice is already part of the optimal solution, and the property holds.

- If $s_j \neq x_k$, it must be that $s_j < x_k$ because x_k is the furthest reachable station by definition.
- We can create a modified sequence of stops, S', by replacing the stop at s_j with a stop at x_k.
- Because $x_k > s_j$, the distance from x_k to the next stop in sequence S (let's call it s_m) is strictly less than the distance from s_j to s_m.
- Since the vehicle could originally reach s_m from s_j, it can certainly reach s_m from x_k (i.e., $s_m - x_k < s_m - s_j \leq L$).
- Therefore, S' remains a valid sequence of stops. Because S' has the exact same number of stops as S, S' is also an optimal solution. This proves the greedy choice is always safe.

Optimal Substructure Property **[1 Mark]**

Property:
If an optimal solution to the overall problem stops at station x_k first, then the remaining sequence of stops must form an optimal solution for the subproblem of traveling from position x_k to the destination D with a full charge.

Question 7

You are given n distinct points $x_1 < x_2 < \ < x_n$ on a line and a positive number d. You may delete some of these points. The objective is to delete the minimum possible number of points such that, among the remaining points, the distance between every pair of consecutive points is at least d. Design an efficient greedy algorithm for this problem. State the greedy-safeness property for your algorithm and prove it. Also state the optimal substructure property.

Answer:

Algorithm: (**1 Marks**)
1. Fix the point x_1.
2. Select the smallest point x_j, such that $x_j \geq x_1 + d$.
3. Delete all the $j-2$ points between x_1 and x_j.
4. Recursively solve the subproblem from x_j to x_n.

Greedy safeness property: (**0.5 Marks**) There exists an optimal solution that makes the greedy choice(i.e. it also deletes the same $j-2$ points).

Proof: (**1.5 Marks**)

Suppose the optimal solution, say O does not make the same choice as greedy, say G. If X^* is the set deleted by greedy choice, then we start by assuming that $X^* \not\subset O$. So, there exists an $x_i \in X^*$ and $x_i \notin O$.

So the optimal solution does not delete x_i, hence it must have deleted x_1, since $x_i \in X^*$ so it is at distance $< d$ from x_1. Now every other non deleted point in O is at a distance d. Hence, x_i and the next non deleted point in O are also at distance d. So, we can replace x_i with x_1 in the optimal solution. Everything else remains the same and we get an optimal solution that makes the greedy choice.

Optimal Substructure Property: (**1 Marks**)

Combining x_1, x_j (greedy choice), with an optimal solution to the problem on x_j to x_n, we get an optimal solution to our problem.

Question 8: Dynamic Programming: Stair Climbing

1. State Definition and Base Cases

(Base Cases – 1 Mark)

Let

$$dp[i]$$

denote the minimum cost required to reach stair i starting from stair 0.

We also maintain an array

$$parent[i]$$

where $parent[i]$ stores the previous stair from which stair i was reached in an optimal solution. This array will be used to reconstruct the optimal path.

We use $+\infty$ to represent an unreachable state.

Since we can move 1, 2, or 3 stairs at a time, the first three states are initialized explicitly.

Base Case: Stair 0

Since we start at stair 0,

$$dp[0] = 0, \qquad parent[0] = -1.$$

No cost is incurred at stair 0.

Base Case: Stair 1

Stair 1 can only be reached directly from stair 0.

Therefore,

$$dp[1] = \begin{cases} c_1, & \text{if stair 1 is available,} \\ +\infty, & \text{if stair 1 is unavailable.} \end{cases}$$

If stair 1 is available, then

$$parent[1] = 0.$$

Base Case: Stair 2

Stair 2 can be reached from either stair 0 or stair 1.

Therefore,

$$dp[2] = \begin{cases} c_2 + \min\{dp[0], dp[1]\}, & \text{if stair 2 is available and a predecessor is reachable,} \\ +\infty, & \text{otherwise.} \end{cases}$$

If stair 2 is reachable, then

$$parent[2] = \arg\min_{j \in \{0,1\}} dp[j].$$

Base Case: Stair 3

Stair 3 can be reached from stair 0, 1, or 2.

Therefore,

$$dp[3] = \begin{cases} c_3 + \min\{dp[0], dp[1], dp[2]\}, & \text{if stair 3 is available and a predecessor is reachable,} \\ +\infty, & \text{otherwise.} \end{cases}$$

If stair 3 is reachable, then

$$parent[3] = \arg\min_{j \in \{0,1,2\}} dp[j].$$

2. Recurrence Relation

(Recurrence – 2 Marks)

For every stair $i \geq 4$, the previous stair must be one of

$$i-1, \qquad i-2, \qquad i-3.$$

Thus, if stair i is available and at least one of its predecessors is reachable,

$$dp[i] = c_i + \min\{dp[i-1], dp[i-2], dp[i-3]\}.$$

More formally,

$$dp[i] = \begin{cases} c_i + \min\limits_{\substack{j \in \{i-1, i-2, i-3\} \\ dp[j] < +\infty}} dp[j], & \text{if stair } i \text{ is available} \\ & \text{and a reachable predecessor exists,} \\ +\infty, & \text{otherwise.} \end{cases}$$

Whenever stair i is reachable, we store the predecessor that gives the minimum value:

$$parent[i] = \arg\min_{\substack{j \in \{i-1, i-2, i-3\} \\ dp[j] < +\infty}} dp[j].$$

If stair i is unavailable, we set

$$dp[i] = +\infty, \qquad parent[i] = -1.$$

Hence, the final minimum cost is

$$\boxed{dp[n]}.$$

If

$$dp[n] = +\infty,$$

then stair n is unreachable.

3. Dynamic Programming Algorithm

The complete algorithm can be written as follows.

Algorithm 4 Minimum Cost Stair Climbing

```
dp[0] ← 0
parent[0] ← −1
for i ← 1 to n do
    dp[i] ← +∞
    parent[i] ← −1
end for
if stair 1 is available then
    dp[1] ← c_1
    parent[1] ← 0
end if
for i ← 2 to n do
    if stair i is available then
        for d ∈ {1, 2, 3} do
            j ← i − d
            if j ≥ 0 and dp[j] < +∞ then
                if dp[j] + c_i < dp[i] then
                    dp[i] ← dp[j] + c_i
                    parent[i] ← j
                end if
            end if
        end for
    end if
end for
if dp[n] = +∞ then
    return "Unreachable"
else
    return dp[n]
end if
```

The loop automatically handles the special cases for stairs 2 and 3, while the explicit initialization of $dp[1]$ handles the first reachable stair.

4. Reconstruction of the Optimal Path

(Reconstruction using Backtracking – 1 Mark)

The minimum cost alone does not give the actual sequence of stairs. Therefore, we use the *parent* array to reconstruct the optimal path.

Starting from stair n, repeatedly follow the parent pointer:

$$n,\ parent[n],\ parent[parent[n]], \ldots, 0.$$

This produces the optimal path in reverse order. We therefore reverse the result to obtain the path from stair 0 to stair n.

Algorithm 5 Reconstruct Optimal Path

```
if dp[n] = +∞ then
    return "Unreachable"
end if
path ← [ ]
cur ← n
while cur ≠ −1 do
    append cur to path
    cur ← parent[cur]
end while
reverse(path)
return path
```

Thus, the final answer contains:

$$\boxed{\text{Minimum Cost} = dp[n]}$$

and

$$\boxed{\text{Optimal Path} = path}.$$

5. Correctness

For any stair i, a valid path reaching i must come from one of the previous three stairs:

$$i-1, \quad i-2, \quad i-3.$$

The recurrence considers all these possibilities and selects the reachable predecessor with minimum cost. The cost c_i of stepping onto stair i is then added.

Therefore, $dp[i]$ represents the minimum possible cost of reaching stair i.

If a stair is unavailable, its DP value is set to $+\infty$, so no optimal path can use that stair.

Since $parent[i]$ stores the predecessor that produced the optimal value of $dp[i]$, following the parent pointers from n reconstructs an optimal path.

6. Complexity Analysis

(Time Complexity – 0.5 Mark)

For every stair, we examine at most three possible previous stairs. Therefore,

$$T(n) = O(3n) = \boxed{O(n)}.$$

(Space Complexity – 0.5 Mark)

We store two arrays, *dp* and *parent*, each containing $n + 1$ elements.

Therefore,

$$S(n) = O(n) = \boxed{O(n)}.$$